

Draft

September 2026

Table 1: API specification availability

API	Successfully Deployed	Specification Obtained Via Repository	Reason for Exclusion
capital	✓	Obtained from the local OpenAPI endpoint. (/openapi.json)	
crapi	✓	Generated from the repo's Postman collection.	
dvapi	✓	Generated from the repo's Postman collection.	
dvws	✓	✓	
Juice-Shop-1	✗	-	No API specification found.
Juice-Shop-2	✗	-	No API specification found.

Continued on next page

Table 1 – continued from previous page

API	Successfully Deployed	Specification Obtained Via Repository	Reason for Exclusion
memos	×	-	Project is no longer runnable. Dependencies have changed. Attempts were made to update the project with current libraries, but without success.
REStaurant	✓	Required changes to the specification generator.	
vampi	✓	✓	
vapi	✓	Obtained from the reDoc interface.	
vuln_bank	✓	Obtained from the local OpenAPI endpoint. (/static/openapi.json)	
vulnerable_rest_api	×	-	No API specification could be found. An attempt was also made to generate one from the Express documentation, but without success.

Of the evaluated APIs, only Vampi and DVAPI matched the original API specification used by Johansens et al. (2025).

Note to mentor: { I have no idea how to put this into zotero. Because it

was found here <https://sites.google.com/utwente.nl/43twentestudentconference/programme-and-proceedings>
 14:00 - 14:15 - Emils Johansens - link to a google docs (https://drive.google.com/file/d/1mqfIWQNqHW_VC2thABM2Ei2AlXvw2BNT/view) }

Table 2: Overview of the evaluated vulnerable API applications

Application	Local URL	OpenAPI / Swagger	Repository
Capital	http://localhost:4100/	http://localhost:8000/docs	GitHub
crAPI	http://localhost:8888	No Swagger endpoint identified	GitHub
DVAPI	http://127.0.0.1:3000	http://127.0.0.1:3000/swagger	GitHub
DVWS	http://localhost:80	http://localhost:80/api-docs	GitHub
RESTaurant	http://localhost:8091/	http://localhost:8091/	GitHub
VAPI	http://127.0.0.1:8000/	http://127.0.0.1:8000/vapi	GitHub
VAmPI	http://127.0.0.1:5002	http://127.0.0.1:5002/ui/	GitHub
vuln-bank	http://localhost:5000/	http://localhost:5000/api/docs/	GitHub

Table 3: RAMA-CLI API Metrics and Evaluation Results

API	NOR	DMR	LP	BRC	SIDC	WSIC	APL	DW	LMLC	APO	Standard	Normalized
Capital	1	0.3559	5	1.0	0.4976	21	3.0	154	0.8102	1.2857	35.29%	50.67%
crAPI	5	0.2192	7	0.4865	0.9587	37	4.7353	99	0.7193	3.1892	47.06%	61.33%
DVAPI	2	0.1064	3	0.9375	0.6500	16	2.1333	2	1.0	0.6875	58.82%	68.67%
DVWS	4	0.2597	5	0.8710	0.0548	31	2.7500	21	0.9715	1.0968	47.06%	61.33%
RESTaurant	12	0.5660	3	0.1905	0.3071	21	1.5000	87	0.9041	0.8571	47.06%	60.67%
VAmPI	5	0.4444	4	0.5714	0.2363	14	2.5000	62	0.7983	0.7143	35.29%	53.33%
vAPI	1	0.2143	5	1.0	0.8762	21	3.7000	29	0.7417	1.9048	41.18%	51.33%
Vulnerable Bank	13	0.3978	5	0.5641	0.0081	39	2.7949	98	0.9774	0.7692	47.06%	60.00%

Metrics Key:

NOR	NumberOfRootPaths	Count of distinct root-level paths.
DMR	DistinctMessageRatio	Ratio of unique messages to total messages.
LP	LongestPath	Length of the longest path segment.
BRC	BiggestRootCoverage	Maximum fraction of operations under a single root path.
SIDC	ServiceInterfaceDataCohesion	Cohesion metric of parameter types in operations.
WSIC	WeightedServiceInterfaceCount	Total count of different operations.
APL	AveragePathLength	Mean segment length of all paths.
DW	DataWeight	Data-related complexity metric.
LMLC	LackOfMessageLevelCohesion	Evaluation of message level dissimilarity.
APO	ArgumentsPerOperation	Average number of parameters/arguments per operation.

Table 4: Selected vulnerable APIs and their vulnerable endpoints

API Name	Vulnerable Endpoints
Capital	DELETE /articles/{slug}/comments/{commentId}
crAPI	GET /identity/api/v2/vehicle/{vehicleId}/location GET /workshop/api/mechanic/mechanic_report
dvAPI	GET /api/getNote
dvws	/api/v2/notes/{ID}
RESTaurant	GET /profile
VAmPI	GET /books/v1/:book_title PUT /users/v1/:username/password PUT /users/v1/:username/email GET /users/v1/:username GET /users/v1 GET /users/v1/_debug
vapi	GET /vapi/api1/user/{api1_id}
vuln-bank	GET /transactions/{account_number}

Source: The vulnerable APIs and the BOLA vulnerabilities were obtained from the Vulnerable-APIs repository: <https://github.com/ejohansens/Vulnerable-APIs>.

Among the BOLA vulnerabilities identified by Johansens et al., all vulnerabilities were manually verified to confirm that they were present. This was true for all, with the exception of one VAmPI endpoint.

Manual testing showed that the endpoint `PUT /users/v1/:username/email` does not actually modify the user specified in the URL. Instead, it modifies the account associated with the authentication token used to make the request.

For example, suppose that user `userA` attempts to change the email address of `userB` by using `userA`'s access token:

```
curl -X PUT http://127.0.0.1:5002/users/v1/userb/email \
-H "Authorization: Bearer $USERA_TOKEN" \
-H 'Content-Type: application/json' \
-d '{
  "email": "YourEmailHasBeenModified@example.com"
}'
```

Although the request appears to target `userB` through the `:username` parameter in the URL, the email address of `userB` is not actually modified. This can be verified by using `userB`'s token and calling the `/me` endpoint.

The reason for this behavior can be found in the implementation of the endpoint. The user is retrieved using the following statement:

```
user = User.query.filter_by(username=resp['sub']).first()
```

Here, `resp['sub']` contains the username associated with the authenticated user, as specified by the access token. As a result, the application does not use the `username` supplied in the URL to determine which user should be modified.

Therefore, if `userA` sends a request targeting `/users/v1/userb/email`, the application still retrieves `userA` from the database and modifies `userA`'s email address. The `userB` value in the URL has no effect on the database query. This behavior means that the endpoint is not vulnerable to BOLA.

So besides `PUT /users/v1/:username/email` the vulnerabilities mentioned in the table form the ground truth for the comparison study.

Table 5: Comparison of vulnerable endpoint detection by Authprobe and ASTF

API	Vulnerable Endpoint	Authprobe	ASTF
Capital	DELETE/articles/{{slug}}/comments/{{commentId}}	WIP	WIP
crAPI	GET/identity/api/v2/vehicle/{vehicleId}/location	×	×
	GET/workshop/api/mechanic/mechanic_report	×	×
dvAPI	GET/api/getNote	×	WIP
dvws	/api/v2/notes/{ID}	×	WIP
RESTaurant	GET/profile	×	WIP
VAmPI	GET/books/v1/:book_title	✓	✓
	PUT/users/v1/:username/password	×	✓
	GET/users/v1/:username	✓	×
	GET/users/v1	×	×
	GET/users/v1/_debug	×	×
vapi	GET/vapi/api1/user/{api1_id}	WIP	WIP
vuln-bank	GET/transactions/{account_number}	×	WIP
	}		

Note: A checkmark (✓) indicates that the tool detected the vulnerable endpoint, while a cross (×) indicates that the vulnerability was not detected. “WIP” indicates that the endpoint has not yet been tested with the respective tool.